

LABORATORIO # 8

Manuel Martinez y Sofi Lam

Problema #1:

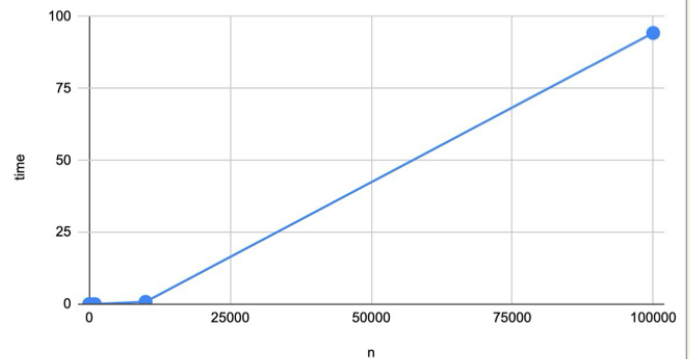
1 a) Note que el ciclo más interno equivale a realizar incrementos de uno hasta $\log_2 n$.

$$\text{Entonces se realizan } \sum_{i=1}^n \sum_{j=1}^{n/2} \sum_{k=0}^{\log_2 n} 1 = \frac{n}{2} \cdot \frac{n}{2} \cdot \log_2 n = \frac{1}{4 \log 2} \cdot n^2 \log n = O(n^2 \log n)$$

b)

n	ops	time	O(n)	O(n)/ops	O(n)/time
1	2	0.000003	0	0	0
10	120	0.000002	100	0.8333333333	50000000
100	17850	0.000042	20000	1.120448179	476190476.2
1000	2505000	0.005593	3000000	1.19760479	536384766.7
10000	350070000	0.809769	400000000	1.142628617	493968032.9
100000	42500850000	94.297427	50000000000	1.176447059	530237161.2
Teorico			1.204119983		

time vs. n



Problema #2:

a)

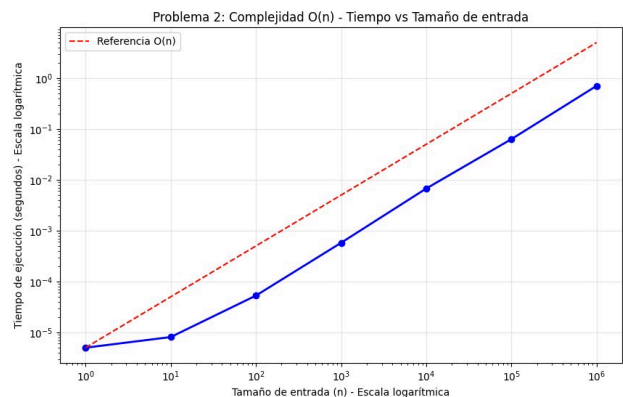
```
1 void function(int n) {
2   if (n <= 1) return;
3   int i, j;
4   for (i = 1; i <= n; i++) {
5     for (j = 1; j <= n; j++) {
6       printf("Sequence\n");
7       break;
8     }
9   }
10 }
```

Nótese que el bucle interno "for" se ejecuta una vez en cada iteración, mientras que el "for" externo se ejecuta n veces. $\Rightarrow$ tiene complejidad $O(n)$.

Figure 1

b)

=== PROFILING PROBLEMA 2 ===		
n	Tiempo (s)	Operaciones
1	0.000005	1
10	0.000008	10
100	0.000053	100
1000	0.000578	1000
10000	0.006756	10000
100000	0.063174	100000
1000000	0.705657	1000000



Problema #3:

a)

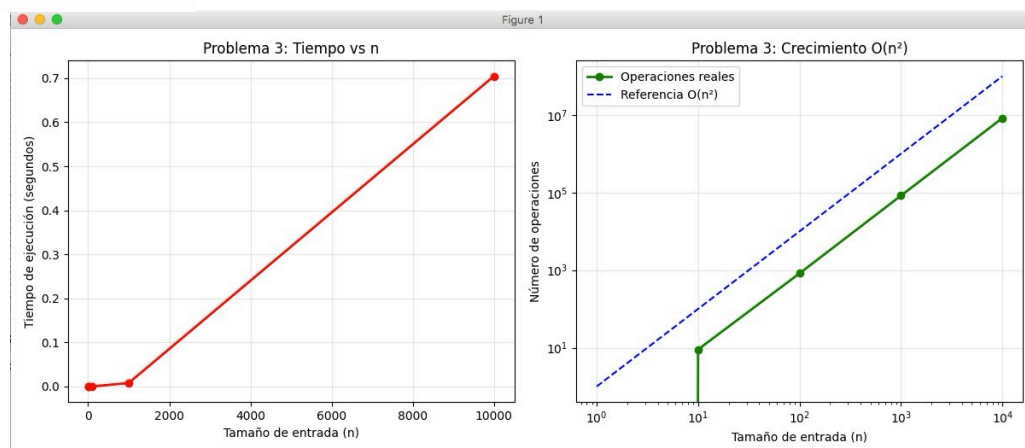
```
1 void function(int n) {
2   int i, j;
3   for (i = 1; i <= n/3; i++) {
4     for (j = 1; j <= n; j += 4) {
5       printf("Sequence\n");
6     }
7   }
8 }
```

Nótese que el bucle "for" interno tiene un incremento de 4 en cada iteración, entonces, tiene aproximadamente $n/4$ iteraciones. Mientras que para el bucle externo, $i \leq n/3$, entonces tiene aproximadamente $n/3$ iteraciones $\Rightarrow (\frac{n}{4})(\frac{n}{3}) = \frac{n^2}{12} \Rightarrow$ su complejidad es $O(n^2)$.

b)

=== PROFILING PROBLEMA 3 ===

n	Tiempo (s)	Operaciones
1	0.000006	0
10	0.000010	9
100	0.000139	825
1000	0.008702	83250
10000	0.744301	8332500



Problema #4:

1. Búsqueda lineal:

- a) Mejor caso: el elemento buscado está en la primera posición $\Rightarrow O(1)$
- b) Caso promedio: el elemento buscado está en alguna posición intermedia, y toma $n/2$ comparaciones encontrarlo, entonces $O(n)$
- c) Peor caso: el elemento buscado está en la última posición, o no existe $\Rightarrow$ toma n comparaciones $\Rightarrow O(n)$

2. Búsqueda binaria:

- a) Mejor caso: el elemento buscado está exactamente en medio, por lo que toma 1 comparación $\Rightarrow O(1)$
- b) Caso promedio: el elemento buscado está en alguna posición intermedia, por lo que toma aproximadamente $\log_2(n)$ comparaciones $\Rightarrow O(\log n)$.
- c) Peor caso: el elemento buscado está en los extremos, o no existe $\Rightarrow$ toma $\log_2(n)$ comparaciones $\Rightarrow O(\log n)$.

3. Quick Sort:

- a) Mejor caso: el pivote divide el array en 2 mitades iguales $\Rightarrow$ altura del árbol de recursión: $\log n$, y el trabajo por nivel: $n \Rightarrow O(n \log n)$
- b) Caso promedio: el pivote divide el array en n partes balanceadas $\Rightarrow O(n \log n)$
- c) Peor caso: el pivote es el elemento más pequeño, o más grande $\Rightarrow O(n^2)$

Problema #5:

5 a)

Se tiene existen $n_0, n_1, c_0, c_1, c_2, c_3 > 0$ tales que $c_0 |f(n)| \leq |g(n)| \leq c_1 |f(n)|$, $n \geq n_0$ y $c_1 |g(n)| \leq |h(n)| \leq c_2 |g(n)|$, $n \geq n_1$. Entonces, para $n \geq n_2 := \max\{n_0, n_1\}$, $c_1^{-1} |h(n)| \leq |g(n)| \leq c_0 |f(n)|$, y $c_0 |f(n)| \leq |g(n)| \leq c_1^{-1} |h(n)|$ y basta escoger $C_2 := (c_0 c_1)^{-1}$, $c_2 := (c_0 c_1)^{-1}$. $\checkmark$

b)

Se tiene existen $n_0, n_1, c_0, c_1 > 0$ tales que $|f(n)| \leq c_0 |g(n)|$, $n \geq n_0$ y $|g(n)| \leq c_1 |h(n)|$, $n \geq n_1$. Entonces, para $n \geq n_2 := \max\{n_0, n_1\}$, $|h(n)| \geq c_1^{-1} |g(n)| \geq (c_0 c_1)^{-1} |f(n)|$, y basta escoger $C_2 := (c_0 c_1)^{-1}$. $\checkmark$

c)

Note que se tienen tantas operaciones como $\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-1} (n-i) = n^2 - \sum_{i=1}^n i + n = n^2 - \frac{n(n+1)}{2} + n = \frac{n^2}{2} + \frac{n}{2}$

Además, para $n \geq 1$, $n^2 \geq n$ y $\frac{n^2}{2} + \frac{n}{2} \leq \frac{n^2}{2} + \frac{n^2}{2} = n^2$. Entonces $f(n) = O(n^2)$. $\checkmark$